# Lecture 02 Worksheet
# MLIR IR Basic Structure

| | |
|---|---|
| 목표 | Operation/Value/Type/Attribute 를 구분하고 NPU lowering 관점에서 해석한다. |
| 제출물 | IR annotation 표, def-use chain 표, NPU op 설계 초안 |
| 권장 시간 | 개인 20 분 + 팀 토론 10 분 + 리뷰 10 분 |

## Exercise 1. Operation Anatomy Annotation

**다음 operation 을 구성요소별로 표시하세요.**

```
%out, %token = "npu.matmul"(%A, %B, %acc) {
  tile = [16, 16, 64], memory_space = "sram", acc_type = i32
} : (memref<128x64xf16>, memref<64x128xf16>, memref<128x128xi32>)
    -> (memref<128x128xi32>, !npu.token) loc("kernel.mlir":12:3)
```

| 구성요소 | 찾은 내용 | NPU compiler 의미 |
|---|---|---|
| Results | | |
| Operation name | | |
| Operands | | |
| Attributes | | |
| Operand types | | |
| Result types | | |
| Location | | |

## Exercise 2. Def-Use Chain

**다음 IR 에서 SSA value 의 definition 과 use 를 표로 작성하세요.**

```
module {
  func.func @matmul_basic(
      %lhs: tensor<4x8xf32>,
      %rhs: tensor<8x16xf32>) -> tensor<4x16xf32> {
    %zero = arith.constant 0.000000e+00 : f32
    %init = tensor.empty() : tensor<4x16xf32>
    %filled = linalg.fill ins(%zero : f32)
              outs(%init : tensor<4x16xf32>) -> tensor<4x16xf32>
    %out = linalg.matmul
        ins(%lhs, %rhs : tensor<4x8xf32>, tensor<8x16xf32>)
        outs(%filled : tensor<4x16xf32>) -> tensor<4x16xf32>
    func.return %out : tensor<4x16xf32>
  }
}
```

| Value | 정의 op | Type | 사용 op | NPU lowering 메모 |
|---|---|---|---|---|
| %lhs | | | | |
| %rhs | | | | |
| %zero | | | | |

| Value | 정의 op | Type | 사용 op | NPU lowering 메모 |
|---|---|---|---|---|
| %init | | | | |
| %filled | | | | |
| %out | | | | |

## Exercise 3. Attribute vs Operand Decision

아래 항목을 attribute 로 둘지 operand 로 둘지 결정하고 이유를 적으세요.

| 항목 | Attribute? | Operand? | 이유 |
|---|---|---|---|
| tile size [16,16,64] | | | |
| runtime input batch size | | | |
| quant scale for static INT8 weight | | | |
| activation pointer address | | | |
| SRAM memory space | | | |
| DMA dependency token | | | |

## Exercise 4. Custom NPU Operation Draft

npu.matmul, npu.dma, npu.barrier 중 하나를 선택해 generic assembly 스타일 pseudo op 를 작성하세요.

**템플릿**

```
%result = "npu.YOUR_OP"(%operand0, %operand1) { key = value } : (...) -> (...)
```

**Operation name:** ____________________________________________________________

**Operands:** ____________________________________________________________

**Results:** ____________________________________________________________

**Types:** ____________________________________________________________

**Attributes:** ____________________________________________________________

**Verifier rule:** ____________________________________________________________

**Expected lowering target:** ____________________________________________________________

# Answer Sketch / Instructor Notes

**주의**

아래는 정답이라기보다 리뷰 가이드입니다. custom NPU dialect 설계에는 여러 타당한 선택지가 있을 수 있습니다.

## Exercise 1 guide

- Results: %out, %token
- Operation name: npu.matmul
- Operands: %A, %B, %acc
- Attributes: tile, memory_space, acc_type
- Types: operands are memref types, results are memref and !npu.token
- Location: loc("kernel.mlir":12:3)

## Exercise 2 guide

- %lhs/%rhs are function block arguments and used by linalg.matmul.
- %zero is defined by arith.constant and used by linalg.fill.
- %init is defined by tensor.empty and used by linalg.fill.
- %filled is defined by linalg.fill and used as output init for linalg.matmul.
- %out is defined by linalg.matmul and used by func.return.

## Exercise 3 guide

- Static tile size, static weight quant scale, and memory space are natural attributes.
- Runtime batch size and pointer address are usually operands or runtime descriptors.
- DMA dependency token is a value/result if it controls ordering between commands.